Linux Troubleshooting — Scenario #6: "The Pipeline Can't SSH In"

DevOps on-call runbook • Syed Aftab • Ubuntu (WSL) • sshd + journalctl + namei • Reference for real incidents & interviews

The alert (7:00 AM): the deploy pipeline has been failing since last night with `Permission denied (publickey)`. The SSH key hasn't changed.
Root cause found: a teammate ran `chmod -R 777 /home/deploy` to copy build files in. sshd's **StrictModes** check then refused to trust `authorized_keys`, because anyone could have edited it. **Fix:** restore `750` / `700` / `600` on home / .ssh / authorized_keys.

The opposite of Scenario #5: there the app had **too little** permission. Here the folder had **too much**, and sshd refused the key **on purpose**. The client only ever says "Permission denied"; **the reason is only in the server's logs.**

The methodology

#	Phase	Commands	Goal
1	Confirm	`ssh -o BatchMode=yes -i KEY user@host`	Reproduce exactly what the pipeline sees
2	Locate (client)	`ssh -v ...`	Network OK? Right server? Right key offered? Where does it stop?
3	Identify (server)	`journalctl -u ssh`, `namei -l`	The real reason, then every level of the path
4	Fix	`chmod 750 / 700 / 600` level by level	Exact permissions, not another recursive chmod
5	Verify	`ssh -o BatchMode=yes` + `journalctl | grep Accepted`	Login works, confirmed on both sides
6	Prevent	shared folder + group, config management, alerts	Give teammates what they need without breaking SSH

SETUP Building the lab (healthy baseline first)

0a. Is the SSH server running? — socket activation

```
sudo systemctl status ssh
   Loaded: ... ssh.service; disabled
   Active: inactive (dead)
TriggeredBy: * ssh.socket
systemctl status ssh.socket
   Active: active (listening)
   Listen: 0.0.0.0:22 (Stream)
           [::]:22 (Stream)
```

Clue	Meaning
`ssh` / `sshd`	the SSH **server** service: `ssh` on Ubuntu, `sshd` on RHEL/Rocky. Package: `openssh-server` (the `ssh` command is the client)
`inactive (dead)` but `TriggeredBy: ssh.socket`	**socket activation**: systemd itself listens on port 22 and starts ssh.service on demand when a connection arrives
`active (listening)`, `Listen 0.0.0.0:22 / [::]:22`	port 22 on all IPv4 and IPv6 addresses (like nginx's listen 80 / [::]:80)
`Tasks: 0`	no sshd running right now — normal with socket activation

Lesson: `ssh.service inactive (dead)` does **not** mean SSH is broken on newer Ubuntu. Check the socket and `ss -tlnp | grep :22` (process shows as systemd) before "fixing" something that works.

0b. The deploy user

```
sudo useradd --create-home --shell /bin/bash deploy
ls -ld /home/deploy
drwxr-x--- 2 deploy deploy 4096 Nov  2 07:00 /home/deploy      <-- baseline: 750
```

Part	Meaning
`--create-home`	create /home/deploy — SSH looks for allowed keys inside the user's home
`--shell /bin/bash`	a real shell: the pipeline runs commands after logging in (unlike the nologin service user in #5)
home files	copied from `/etc/skel` (the template for every new home): .bashrc, .profile, .bash_logout

0c. The pipeline's key pair (created as your user — you play the pipeline)

```
ssh-keygen -t ed25519 -f ~/.ssh/deploy_key -N "" -C "ci-pipeline"
The key fingerprint is:
SHA256:vDPle6dedfbJDaZL2vs7f9Dp6bFLc86ZIaHF/x9EYts ci-pipeline
```

Part	Meaning
`ssh-keygen`	create an SSH key pair
`-t ed25519`	key **t**ype: modern, short, strong
`-f ~/.ssh/deploy_key`	**f**ile name → creates deploy_key (private) and deploy_key.pub (public)
`-N ""`	empty passphrase: pipelines run unattended. Personal keys normally get one
`-C "ci-pipeline"`	**c**omment label inside the public key
fingerprint `SHA256:...`	the key's short unique ID — sshd logs it on login, so you can match logins to keys
randomart	a picture of the fingerprint for humans; can be ignored

File	Lives on	Rule
`deploy_key`	the **client** (pipeline)	**private — never share, never copy to a server**. ssh-keygen sets it to 600
`deploy_key.pub`	the **server**	public — safe to share. 644

0d. Authorize the key, then test

```
sudo mkdir -p /home/deploy/.ssh
sudo cp ~/.ssh/deploy_key.pub /home/deploy/.ssh/authorized_keys
sudo chown -R deploy:deploy /home/deploy/.ssh
sudo chmod 700 /home/deploy/.ssh
sudo chmod 600 /home/deploy/.ssh/authorized_keys
ssh -i ~/.ssh/deploy_key -o StrictHostKeyChecking=accept-new deploy@localhost 'whoami; hostname'
deploy
SYEDLENOVO
```

Part	Meaning
`.ssh`	hidden folder (leading dot). `ls -a` shows hidden files
`authorized_keys`	the list of public keys allowed to log in as this user, one per line
`chown -R deploy:deploy`	created with sudo = owned by root → give the folder **and** contents (-R) to deploy (the #5 trap)

Part	Meaning
`chmod 700 .ssh` / `600 authorized_keys`	only deploy can enter / read-write. **sshd requires this**
`-i KEY`	**i**dentity file: log in with this private key
`-o StrictHostKeyChecking=accept-new`	auto-trust a **new** server's host key; still refuse if a known server's key **changed**
`'whoami; hostname'`	run these commands and log out instead of opening a shell — how pipelines use SSH

0e. The overnight change

```
sudo chmod -R 777 /home/deploy        # a teammate's "quick fix" to copy build files in
```

STEP 1 Confirm — reproduce what the pipeline sees

```
ssh -i ~/.ssh/deploy_key deploy@localhost 'whoami'
deploy@localhost's password:          <-- Ctrl+C
ssh -o BatchMode=yes -i ~/.ssh/deploy_key deploy@localhost 'whoami'
deploy@localhost: Permission denied (publickey,password).
```

Clue / part	Meaning
password prompt appeared	SSH tries methods **in order**: the key was refused, so it fell back to a password
`-o BatchMode=yes`	never prompt for anything, just fail — how CI/CD and scripts run, so jobs fail fast instead of hanging
`(publickey,password)`	the methods the server **accepts** — **not** the reason it failed

STEP 2 Locate — the client side with ssh -v

```
ssh -v -o BatchMode=yes -i ~/.ssh/deploy_key deploy@localhost 'whoami'
debug1: Host 'localhost' is known and matches the ED25519 host key.
debug1: Authentications that can continue: publickey,password
debug1: Offering public key: /home/syedaftab04/.ssh/deploy_key ED25519 SHA256:vDPle6dedf...
debug1: Authentications that can continue: publickey,password
debug1: No more authentication methods to try.
deploy@localhost: Permission denied (publickey,password).
```

Part / line	Meaning
`-v`	**v**erbose: every step of the login as debug1 lines (`-vv`, `-vvv` = more)
`-o`	set an **o**ption
`Host ... matches the ED25519 host key`	the **server's** identity is fine — not a rebuilt/changed server
`Offering public key: ... SHA256:vDPle6...`	the client sent **the right key** (fingerprint matches 0c)
the methods list repeated	**the server refused the key** — no reason given
`No more authentication methods`	BatchMode turned off the password fallback

Client-side conclusion: network OK, right server, right key offered — the server rejected it. Everything the client can know stops here. The reason is only on the server.

STEP 3 Identify — the server's logs, then every level of the path

```
journalctl -u ssh --since "10 minutes ago" --no-pager
sshd-session[56127]: Authentication refused: bad ownership or modes for file
                     /home/deploy/.ssh/authorized_keys
sshd-session[56127]: pam_unix(sshd:auth): authentication failure; ... user=deploy
sshd-session[56127]: Failed password for deploy from 127.0.0.1 port 37630 ssh2
sshd-session[56127]: Connection closed by authenticating user deploy ... [preauth]
sshd-session[56277]: Authentication refused: bad ownership or modes for file ...
```

Part / line	Meaning
`-u ssh`	the SSH server's unit (with socket activation, sshd still logs under ssh.service)
`--since "10 minutes ago"`	relative time window
`--no-pager`	print straight out, wrap long lines
`sshd-session[PID]`	newer OpenSSH runs one sshd-session process **per connection**: one PID = one login attempt
`Authentication refused: bad ownership or modes`	**the cause**. modes = permissions. sshd found authorized_keys unsafe and ignored it
`pam_unix ... authentication failure` / `Failed password`	the password typed at the prompt in Step 1 (PAM checks passwords; deploy has none)
`[preauth]`	the connection closed before login completed

Key lesson: for SSH problems the answer is almost always in the **server's** `journalctl -u ssh` (or `/var/log/auth.log` on systems without the journal). The client hides the reason on purpose — telling an attacker exactly what's wrong would help them.

```
sudo namei -l /home/deploy/.ssh/authorized_keys
drwxr-xr-x root   root   /
drwxr-xr-x root   root   home
drwxrwxrwx deploy deploy deploy
drwxrwxrwx deploy deploy .ssh
-rwxrwxrwx deploy deploy authorized_keys
```

Path	Found	sshd needs	Risk
`/home/deploy`	`777`	not group/other-writable (750 or 755)	anyone can create or delete files, even replace .ssh
`.ssh`	`777`	`700`	anyone can replace authorized_keys
`authorized_keys`	`777`	`600`	anyone can **add their own key** and log in as deploy. Also wrongly executable

Owner was `deploy:deploy` everywhere — this time only the **permissions** were wrong (in #5 it was the owner).

Why 777 breaks SSH (StrictModes): authorized_keys decides who can log in as deploy. If others can write to it, the folder above it or the home folder, any user could add their own key. So sshd's **StrictModes** check (on by default) refuses to trust the file. Too much permission is the problem. Other programs refuse loose permissions on purpose too: the ssh client (private key readable by others), MySQL config, sudoers.

STEP 4 Fix — exact permissions, level by level

```
sudo chmod 750 /home/deploy
sudo chmod 700 /home/deploy/.ssh
sudo chmod 600 /home/deploy/.ssh/authorized_keys
sudo chmod 644 /home/deploy/.bashrc /home/deploy/.profile /home/deploy/.bash_logout
sudo chmod 700 /home/deploy/.cache
```

Command	Result	Why
`chmod 750 /home/deploy`	`rwxr-x---`	back to the Step 0b baseline
`chmod 700 .ssh`	`rwx------`	only deploy can enter
`chmod 600 authorized_keys`	`rw-------`	only deploy can read/change the key list; no x
`chmod 644 .bashrc .profile .bash_logout`	`rw-r--r--`	these run automatically as deploy — world-writable = anyone can make deploy run their command
`chmod 700 .cache`	`rwx------`	private cache folder

Don't fix chmod -R with chmod -R. One recursive command can't set folders to 700 and files to 600. Set each level explicitly, and check the whole home with `sudo ls -la /home/deploy` (`-a` shows hidden files; `.` = the folder itself, `..` = its parent).

Dotfile	Runs when	Typical contents
`.profile`	once at **login** (SSH, console)	environment variables, PATH; on Ubuntu also loads .bashrc
`.bashrc`	every **interactive** Bash shell (each new terminal)	aliases (ll, la), prompt, history settings
`.bash_logout`	when a login shell **exits**	cleanup (Ubuntu: clear the screen)
`.cache/`	—	programs' cached data

Real-world tie-in: cron doesn't read .profile/.bashrc — the classic "works over SSH, fails in cron" bug (PATH and variables missing). Use full paths or set variables inside the script.

STEP 5 Verify — both sides

```
ssh -o BatchMode=yes -i ~/.ssh/deploy_key deploy@localhost 'whoami'
deploy
journalctl -u ssh --since "5 minutes ago" --no-pager | grep -E "Accepted|refused"
sshd-session[56914]: Accepted publickey for deploy from 127.0.0.1 port 58544 ssh2:
                     ED25519 SHA256:vDPle6dedfbJDaZL2vs7f9Dp6bFLc86ZIaHF/x9EYts
```

Part	Meaning
`grep -E`	**E**xtended patterns, which allow `|`
`"Accepted|refused"`	lines containing Accepted **or** refused
`Accepted publickey ... SHA256:vDPle6...`	the server accepted the login, with the **same fingerprint** as the pipeline's key

Ticket summary: A teammate ran chmod -R 777 /home/deploy overnight. sshd StrictModes refused authorized_keys ("bad ownership or modes"), so the pipeline's key was rejected. Restored 750 home / 700 .ssh / 600 authorized_keys, and 644 dotfiles. Login verified: Accepted publickey for deploy with the pipeline key fingerprint.

STEP 6 Prevent

1. Give the teammate what they actually needed — a shared folder, not someone's home:

```
sudo groupadd builds
sudo usermod -aG builds deploy
sudo usermod -aG builds TEAMMATE
sudo install -d -o deploy -g builds -m 2775 /srv/builds
```

Part	Meaning
`groupadd builds`	a group for people who share build files
`usermod -aG builds USER`	**a**ppend the user to the **G**roup (without -a, other groups are removed)
`install -d -o -g -m`	create the folder with owner, group and mode in one step (#5)
`2775`	the leading **2** = **setgid**: new files inside automatically get the folder's group, so everyone in builds can share them. 775 = owner + group rwx, others r-x

2. Never chmod -R 777, and never on a home folder — it breaks SSH and makes every dotfile an attack path.

3. Don't "fix" this by turning off StrictModes in sshd_config — that removes the safety check instead of fixing the permissions.

4. Enforce permissions with configuration management — an Ansible task (or similar) that sets `.ssh` to 700 and `authorized_keys` to 600 on every run puts drift back automatically.

5. Alert on it — ship sshd logs to monitoring and alert on `Authentication refused` for service accounts, so you find out before the next deploy fails.

Lab cleanup

```
sudo userdel -r deploy
rm ~/.ssh/deploy_key ~/.ssh/deploy_key.pub
```

Part	Meaning
`userdel -r`	delete the user **and** (-r) its home folder
`rm ~/.ssh/deploy_key*`	remove the lab key pair from your account

REF Full command reference

Command	What	Why here
`systemctl status ssh / ssh.socket`	SSH server and socket state	socket activation: inactive service is normal
`ss -tlnp | grep :22`	who listens on port 22	confirm SSH is reachable
`useradd --create-home --shell /bin/bash U`	create a login user	lab setup
`ssh-keygen -t ed25519 -f F -N "" -C L`	create a key pair	the pipeline's key
`ssh -i KEY user@host 'cmd'`	log in with a key, run a command	how pipelines use SSH
`ssh -o BatchMode=yes ...`	never prompt, just fail	reproduce the pipeline exactly
`ssh -v ...`	verbose client-side steps	right server? right key? refused?
`journalctl -u ssh --since ... --no-pager`	server-side sshd logs	**found the real reason**
`journalctl ... | grep -E "Accepted|refused"`	filter login results	verify the fix on the server

Command	What	Why here
`namei -l PATH`	permissions of every level	**found all three 777 levels**
`ls -la DIR`	all files incl. hidden	found the 777 dotfiles
`chmod 750 / 700 / 600 / 644`	set exact permissions	the fix
`install -d -o U -g G -m 2775 DIR`	shared folder with setgid	prevention
`usermod -aG GROUP USER`	add user to a group	prevention
`userdel -r USER`	delete user + home	cleanup

Interview answer (say this)

"First I reproduce it exactly as the pipeline does, with BatchMode so it can't fall back to a password. ssh -v shows the network is fine, the host key matches and the right key fingerprint is offered, but the server refuses it — and the client never says why. So I go to the server: journalctl -u ssh showed 'Authentication refused: bad ownership or modes' for authorized_keys. namei -l showed the home folder, .ssh and authorized_keys were all 777. sshd's StrictModes refuses keys anyone could have edited. I restored 750, 700 and 600 level by level, fixed the world-writable dotfiles, and verified with a BatchMode login and an 'Accepted publickey' line with the same fingerprint. To prevent it: a shared group folder with setgid instead of opening up someone's home, config management to enforce .ssh permissions, and alerts on refused logins for service accounts."